

Intermediate RAG System Using Pinecone

1. Objective and architecture

This project implements a Retrieval-Augmented Generation application for question answering over PDF documents. Its central objective is traceability: a reader should be able to identify the document, physical page, and passage supporting every displayed answer. The implementation uses Python, Streamlit, pypdf, Sentence Transformers, Pinecone, and the Groq chat-completions API. LangChain is omitted because the relatively small pipeline benefits from explicit control over metadata, retrieval filters, and evidence validation.

The ingestion path is PDF upload, validation, page extraction, text cleanup, token-aware chunking, local embedding generation, and Pinecone upsert. Specific questions use semantic retrieval, with automatic full-text fallback for small documents and broader searches for weak results. Overview requests read all selected extracted text, condensing long selections in batches. Groq produces answer claims with supporting quotations; exact quotation validation and a separate claim-support check precede display. Strict similarity mode retains the original threshold-only retrieval behavior.

Separation of concerns is implemented through small modules. `loader.py` handles file validation and extraction; `chunker.py` handles segmentation; `embeddings.py` owns model loading and encoding; `vector_store.py` encapsulates Pinecone operations; `generator.py` owns the LLM request and grounding checks; `service.py` coordinates ingestion and answering. `app.py` manages user controls and session state. Dataclasses define documents, pages, chunks, retrieval hits, evidence, and answers independently of Streamlit.

The application supports multiple PDFs, each at most 20 MiB. It checks the extension and PDF signature, rejects oversized, damaged, password-protected, and fully non-text PDFs, and preserves original physical page positions. Unicode normalization, whitespace collapse, soft-hyphen removal, and line-break dehyphenation reduce formatting noise. Blank or scanned pages are skipped with a visible warning. OCR is intentionally outside scope; adding it later should preserve page provenance and distinguish recognized text from native extraction.

Five intermediate enhancements are implemented: multi-document collections, session query history, UI chunk-size adjustment, adjustable top-k retrieval, and document/page metadata filtering. History contains at most 20 queries and their results. Each new query is independent: previous generated answers never become retrieval evidence. Similarity is displayed as a relevance measure, without claiming calibrated answer confidence. The interface displays the active namespace and offers deletion of the session's indexed data.

The deployment target is a local educational application. API keys are loaded from environment variables or a local `.env` file; `.env` is excluded from version control. No key is embedded in source, no persistent query log is enabled, and network failures use actionable messages. The application does not implement authentication or shared-user authorization. Uploaded text is stored in Pinecone and relevant excerpts are sent to Groq, so the user should select documents appropriate for these services.

2. Embedding and Pinecone design decisions

The embedding model is sentence-transformers/all-MiniLM-L6-v2. It produces 384-dimensional dense vectors and is used locally on CPU. Both document chunks and questions pass through the same model with normalized output. The model is suitable for an intermediate project because it provides semantic representations without an embedding API key. Its English-oriented retrieval behavior and short input window are limitations that must be evaluated on the intended corpus. The model card describes truncation at 256 word pieces; this design avoids silently embedding only the beginning of a long chunk.

Chunking uses tokenizer offset mappings into cleaned page text. The default chunk budget is 180 content tokens with 30 overlapping tokens, and the UI allows sizes up to 240 tokens. The extra margin leaves room for special tokens within the model limit. When a window is not the last on a page, the splitter searches backward for a sentence-ending token while retaining a useful minimum window length. Long sentences are split at the token limit. Overlap preserves local continuity. Chunks never cross pages, keeping each source reference unambiguous. Using source substrings rather than decoded tokens preserves exact text for subsequent citation validation.

The Pinecone configuration is a serverless dense-vector index with dimension 384 and cosine similarity, using aws/us-east-1 by default. Cloud, region, and index name are configurable. The adapter explicitly creates a missing index, polls readiness, and rejects an existing index with a different dimension or metric. The implementation uses the Pinecone Python SDK 7.x vector API. A dedicated index is recommended because equal dimensions alone do not imply compatible embedding spaces.

Each ingestion creates a fresh UUID namespace. A collection is activated only after all batches have been upserted and Pinecone's index statistics report the expected number of vectors. This accommodates eventual visibility and prevents a failed upload from replacing the previous active collection. Failed staging namespaces remain tracked for cleanup; replaced namespaces are deleted after the new collection becomes active. Namespace count visibility is a practical readiness check, not a transactional guarantee for every search replica. A short transient search delay may still require retrying a question.

Every vector includes chunk_id, document_id, document_name, page, text, and embedding_model metadata. A content hash deduplicates identical PDF uploads; stable chunk IDs incorporate document identity, page, offset, and chunk settings. The service embeds and uploads batches of 64 chunks, while the embedding model processes smaller internal batches. Retrieval always specifies a namespace and filters for active document IDs and the embedding model; optional page filtering adds an equality condition. It requests metadata without vector values, retrieves the selected top-k, and discards matches below the UI threshold.

Namespaces isolate collections but do not expire automatically. Browser state is temporary while Pinecone data persists. The UI provides a session deletion action, and the README explains manual console cleanup when a session is lost. The application never deletes an entire index. This lifecycle is adequate for the assignment; a production version should add authenticated ownership, durable collection manifests, and retention rules.

3. Grounding, error handling, and implementation challenges

The answer generator uses Groq to write concise, source-backed explanations and summaries. GROQ_MODEL defaults to qwen/qwen3.8-27b in non-thinking mode. A live account check showed the earlier llama-3.3-70b-versatile model was unavailable, so configuration was updated. Requests use JSON object mode and a compact output budget suitable for the tested account. Instructions prohibit outside facts and obeying instructions inside source text. English and Roman Urdu questions are supported; source quotations retain their original language. Historical answers are excluded from evidence.

Before display, deterministic validation checks response structure and each claim's citations. Every source ID must belong to the supplied context and every quotation must be an exact contiguous substring of that source. Duplicate evidence is removed. A second model request evaluates each claim against its quotations and surrounding text; all checks must pass. Malformed output, invented quotes, and unsupported claims produce abstention. The exact fallback remains: "The answer is not available in the provided document." Separate diagnostic text explains retrieval or verification limitations.

The interface shows readable answer points followed by supporting excerpts and numbered citations. Citations include the filename, physical page, and cosine score when obtained from semantic search. Directly read text has no invented similarity score. Summary requests bypass similarity matching: generic words such as "summarize this" need not resemble any document passage. Long summaries select validated key quotations from each input batch before composing the overview. Selections exceeding 24 batches require narrower document/page scope, avoiding silently truncated coverage.

These checks cannot prove the absence of hallucination or misleading output. Exact containment ensures quotation provenance; claim support remains a probabilistic judgment by the same model family. The verifier can repeat generation mistakes, miss omitted qualifications, or accept misleading source claims. Condensing documents can lose minor details. Prompt injection remains an evaluation concern. The application reduces risk through bounded outputs, source verification and abstention, but human review is still necessary. Similarity is not a calibrated correctness probability.

The main engineering challenges are preserving citations during preprocessing, respecting the embedding token budget, controlling stale collection data, and distinguishing missing evidence from infrastructure failures. Page-aware chunks solve provenance ambiguity at the cost of losing cross-page context. Tokenizer offsets avoid lossy text reconstruction. Fresh namespaces prevent old and new chunk settings from mixing. The service reports Pinecone and Groq connectivity or quota failures as errors instead of returning the missing-answer fallback. Empty queries, overly long questions, invalid filters, and invalid chunk parameters are rejected early.

Operational limits remain explicit. Large uncompressed PDFs can require substantial memory despite the input size cap. Complex tables and multi-column layouts may extract in an unsuitable order. The first model download increases startup time. Provider model availability and account quotas can change; the model name and Pinecone location are configurable. The current application is designed for coursework rather than hostile public uploads or high-concurrency service operation.

4. Verification and performance analysis

Verification separates local correctness from live service behavior. The expanded offline suite covers PDF validation, chunk coverage, Pinecone SDK request construction, filters, failed staging, summary routing, multi-batch coverage, full-document fallback, generated claims, verification failures and Streamlit interaction. External calls are mocked. A separate MiniLM check produced normalized 384-dimensional vectors and ranked the expected page first with cosine similarity 0.451. Initial loading took 61.55 seconds; encoding the small fixture and query took 0.124 seconds. Current test counts and live regression results are recorded in docs/validation.md.

A reproducible live evaluation script is supplied with a fictional three-page course guide and ten labeled questions. Six have answers on known pages; three request absent information; one attempts to override the instructions. The script loads the real embedding model, indexes the fixture in Pinecone, calls the actual generation service, writes detailed JSON results, and attempts temporary namespace deletion. It aborts on service errors rather than inserting invented results. Results include retrieved pages and individual answer text so reviewers can inspect failures.

The reported metrics are page-level recall at k on answerable questions, expected-answer substring rate, abstention rate on unsupported questions, mean query latency, and ingestion duration. Query latency includes query embedding, retrieval, generation, and evidence validation. Ingestion timing covers extraction through vector visibility but excludes initial model loading. The substring metric is a coarse correctness proxy; it must be supplemented by manual checks of relevance, negation, units, and qualifications. A ten-question fixture is a smoke test and should not be reported as a general accuracy estimate.

The live regression script checks the originally failing summary request, a factual query, a Roman Urdu query, and missing information using a temporary Pinecone namespace. It records case-level answers and timings under artifacts/live_answers.json and removes test vectors. The larger retrieval benchmark defaults to strict mode so automatic full-document fallback does not inflate recall. These small fixtures cannot establish general accuracy. Before submission, extend testing to 30–50 labeled questions across representative PDFs, recording model, settings, cold/warm timing and failures; include unsupported and adversarial questions.

Expected tradeoffs guide that experiment. Raising top-k may improve retrieval coverage while increasing generation context and latency. A higher threshold tends to increase abstention and can discard relevant passages with unusual wording. Smaller chunks can improve local matching but lose qualifications; larger chunks provide more context while mixing topics. Overlap reduces boundary loss but adds vectors and storage. Compare chunk sizes 96, 180, and 240, top-k values 3, 5, and 8, and thresholds 0.25, 0.35, and 0.50 while holding the dataset fixed. Repeat warm queries separately from cold-start runs.

The deliverables include source code, a standalone architecture diagram, this four-page report, an evaluation harness, and a timed six-minute demonstration script. The source is published at github.com/Izharullah733/Pdf-Evidence-RAG. The working application still needs to be recorded to produce the required 5–7 minute video.

Primary references: Pinecone documentation, "Create an index" and "Upsert records" (docs.pinecone.io); the all-MiniLM-L6-v2 model card (huggingface.co/sentence-transformers/all-MiniLM-L6-v2); Groq structured-output documentation (console.groq.com/docs/structured-outputs); and the Streamlit file-uploader API reference (docs.streamlit.io). Direct links are included in README.md.